Comprendre les Transformers

Emmanuel THE - Phillipe Roumbo

Décembre 2025

Contexte et Historique

- Avant : les réseaux récurrents (RNN, LSTM, GRU) ont constitué la base du traitement de séquences
- Ces modèles présentent deux limitations majeures
  - Traitement séquentiel : entraînement lent et difficile à paralléliser.
  - Difficultés avec phrases longues et dépendances à distance

Problématique et Solution

- Solution proposée par l'article *"Attention Is All You Need"* (Vaswani et al., 2017) :
  - Supprimer complètement la récursivité (pas de RNN / LSTM)
  - Utilisation du mécanisme d'attention, appliqué en parallèle

- Architectures entièrement nouvelles grâce à la puissance de calcul decuplée

Impact :

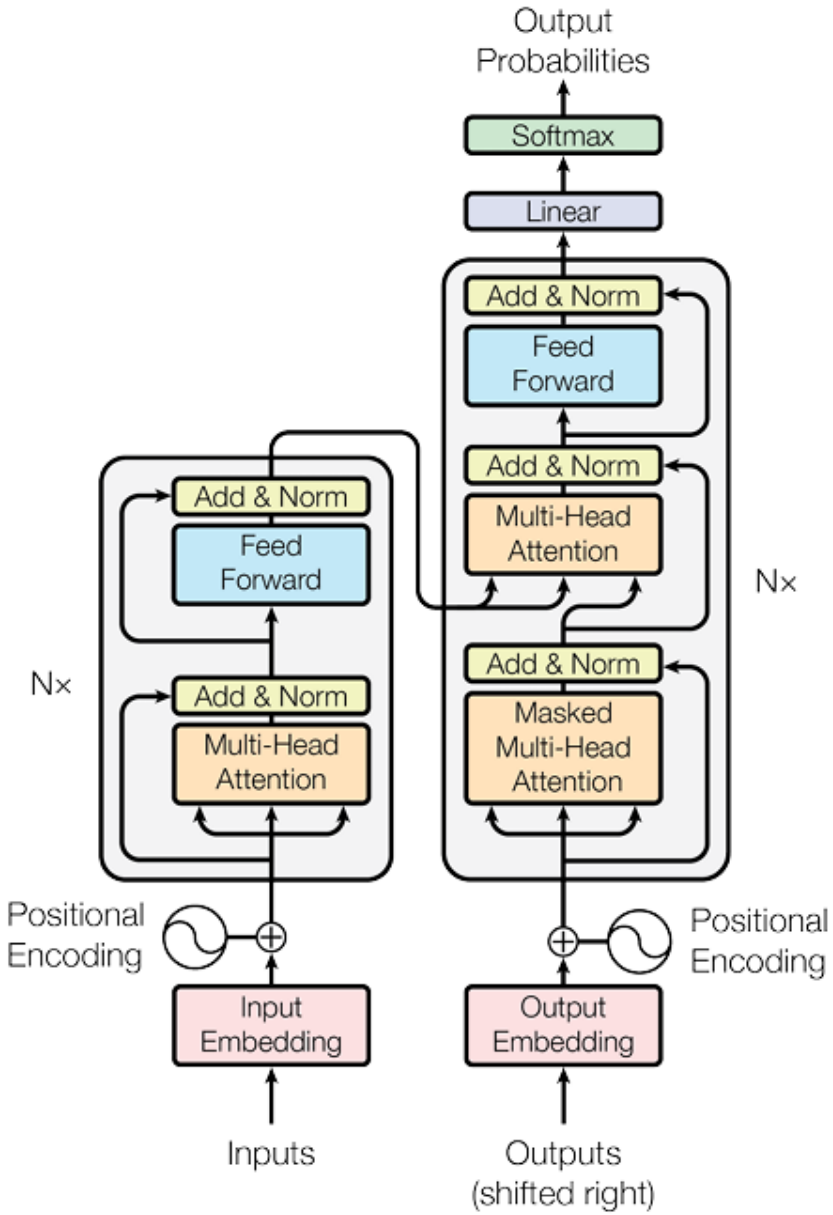
- Apprentissage plus rapide, plus efficace
- Résultats excellents
- Traduction automatique
- Reconnaissance vocale

Architecture du Transformer

Architecture

- Encodeur + Décodeur empilés
- **Encodeur :** Self-Attention multi-têtes + Add & Norm + Feed-Forward + Add & Norm
- **Décodeur :** Self-Attention masquée + Attention encodeur-décodeur + Feed-Forward + Add & Norm
- Positionnal Encoding : injecte l'ordre des mots
- Projection finale + Softmax : génération auto-régressive
- Traite phrases longues, parallélisable, efficace pour le NLP

Architecture du Transformer



Embedding + Positional Encoding

- **Embeddings :** Texte $\rightarrow$ token $\rightarrow$ vecteur d_{model}
- **Positional Encoding :** ajoute la position via sinus/cosinus
- **Utilité :** chaque token encode à la fois son sens et sa position dans la phrase, prêt pour l'encodeur.

Embedding + Positional Encoding

Les Formules Sinus/Cosinus

- *pos* est la position du token ; *i* est l'indice de la dimension du vecteur.

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

- Les longueurs d'onde varient pour permettre au modèle de coder les **positions relatives**.

L'Encodeur : Structure

1. **Multi-Head Self-Attention (Q, K, V)**
   - Permet à chaque mot de "regarder" et **pondérer** l'importance de tous les autres mots.
   - Exécuté en parallèle pour capter différentes relations.
2. **Add & Norm (Résiduelle et Normalisation)**
   - Ajout de l'entrée (connexion résiduelle) puis normalisation.
   - **Rôle :** Stabiliser l'apprentissage et faciliter l'entraînement des réseaux profonds.
3. **Feed-Forward Network (FFN)**
   - Réseau de deux couches appliqué indépendamment à chaque position.
   - **Rôle :** Affiner **localement** la représentation du token après l'étape d'attention.
4. **Add & Norm**
   - Seconde étape de stabilisation pour la sortie du FFN.

L'Encodeur : Self-Attention (Le Cœur Mathématique)

Objectif : Permettre à chaque mot de contextualiser sa propre représentation en pondérant l'importance de tous les autres.

1. Projection en Q, K, V

- La matrice d'entrée **X** est projetée linéairement via des matrices de poids (W^Q, W^K, W^V) :
- **Q** (Query) : Ce que le mot **cherche**.
- **K** (Key) : Comment le mot **peut être trouvé**.
- **V** (Value) : L'**information** qu'il apporte.

L'Encodeur : Self-Attention (Le Cœur Mathématique)

2. Score d'Attention

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- QK^T **:** Produit scalaire pour obtenir les scores de similarité entre chaque Query et toutes les Keys.
- $\sqrt{d_k}$ **:** Mise à l'échelle (scaling) pour éviter les gradients trop petits après l'application du Softmax (stabilisation).
- **Softmax :** Normalise les scores en poids d'importance (somme à 1).
- V **:** Application des poids normalisés aux Values pour obtenir la nouvelle représentation contextuelle.

Architecture du Transformer

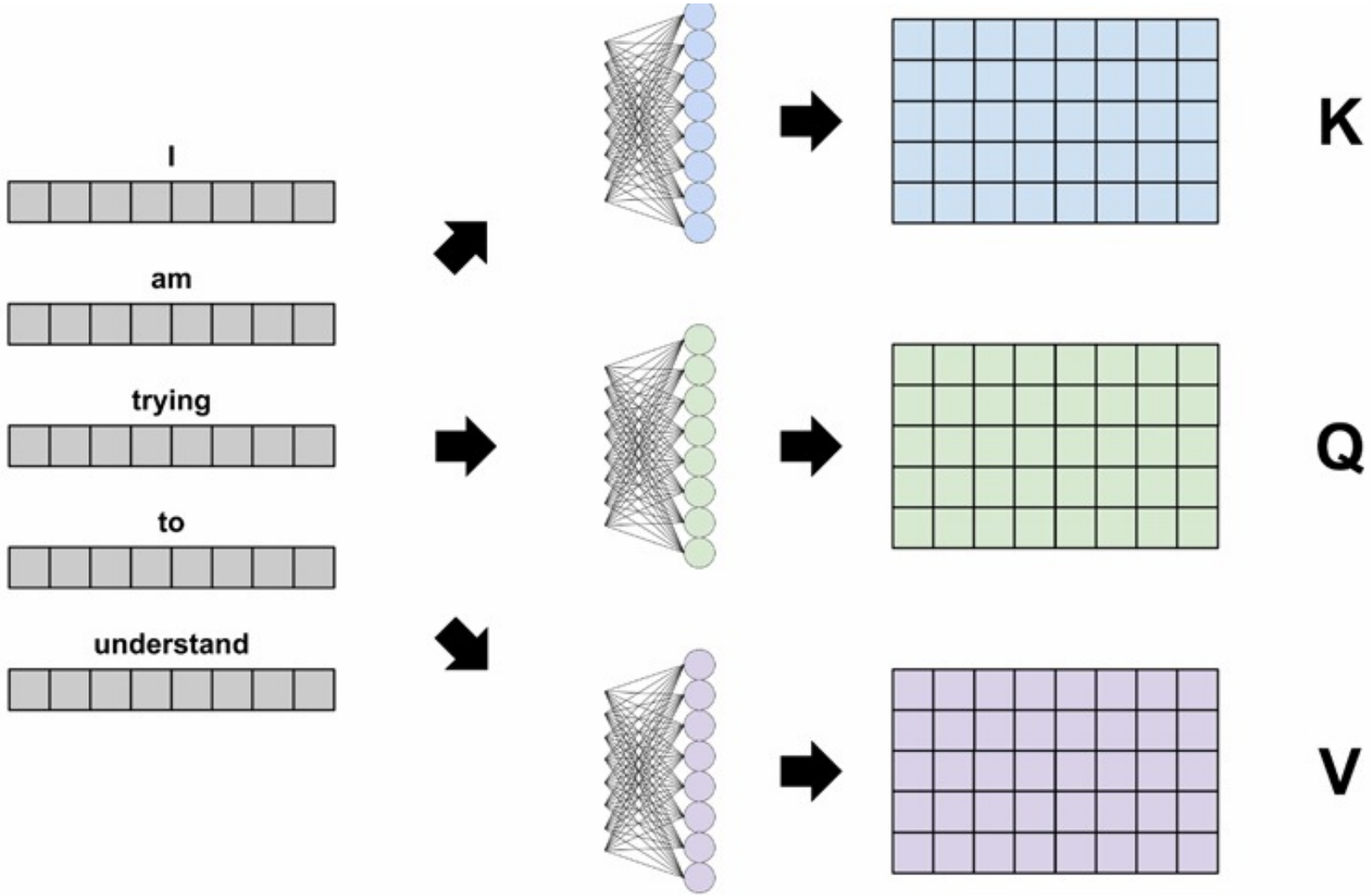


Figure: Explication visuelle de l'Attention

Architecture du Transformer

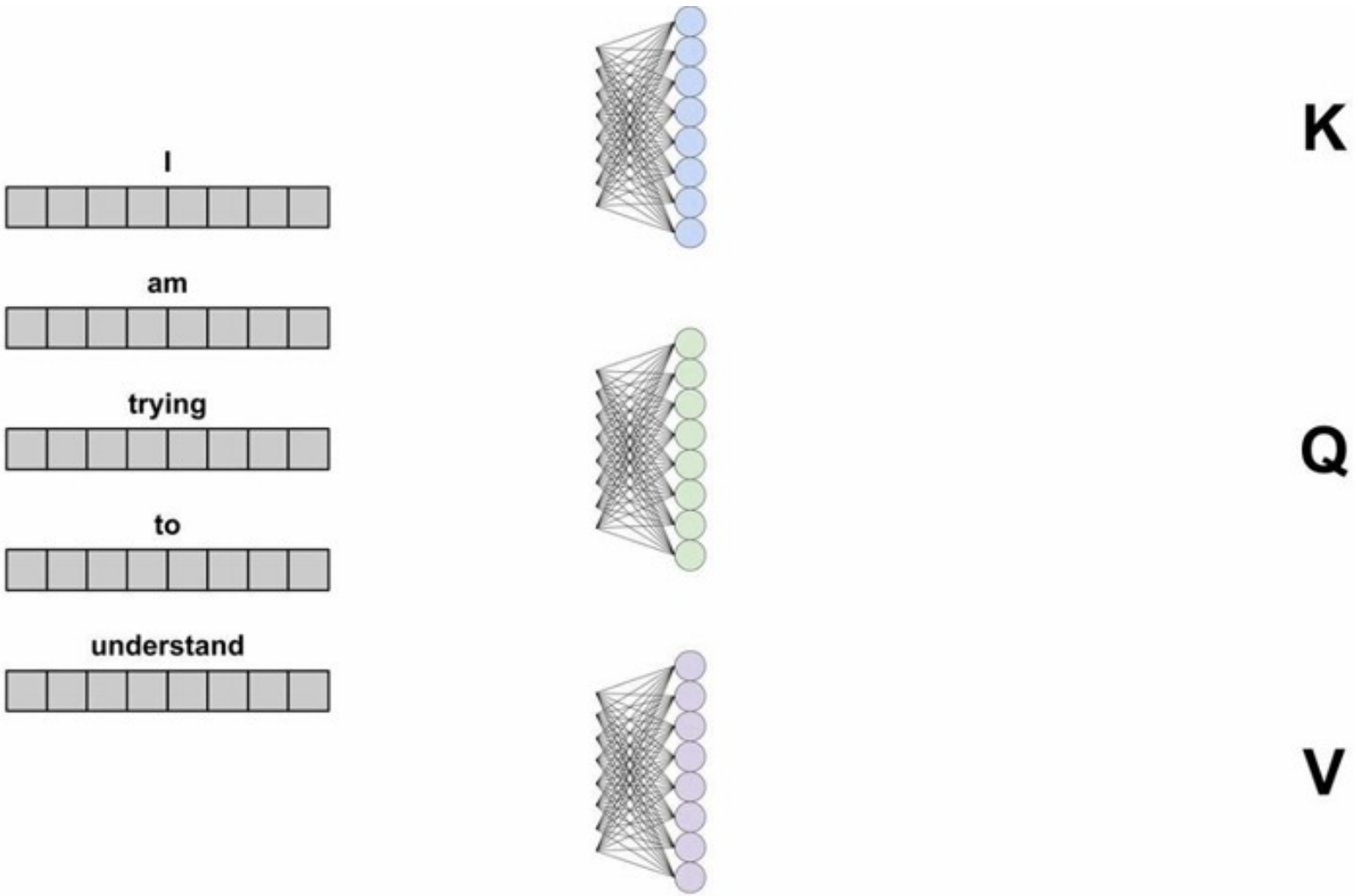


Figure: Explication visuelle de l'Attention

Architecture du Transformer

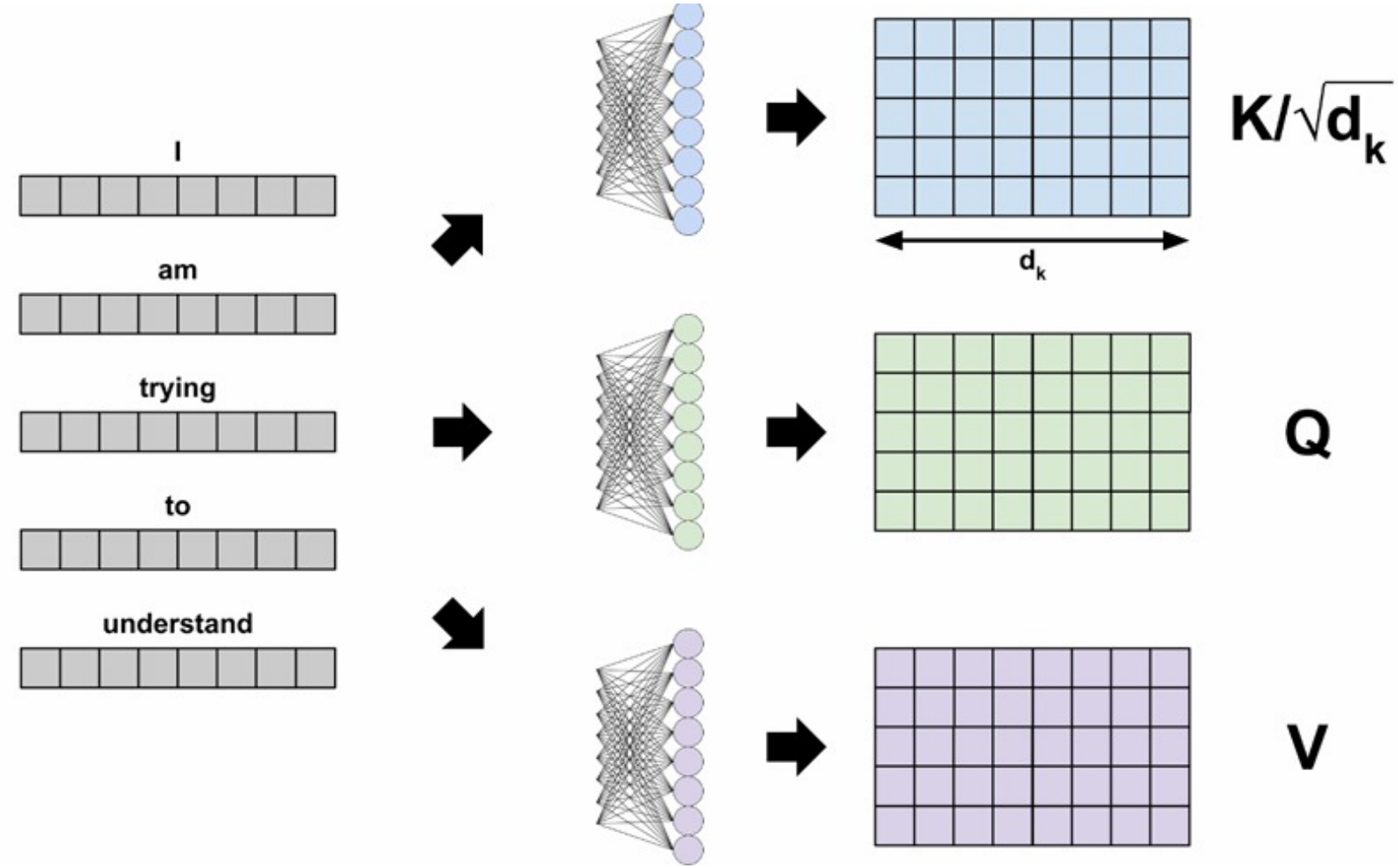


Figure: Explication visuelle de l'Attention

Architecture du Transformer

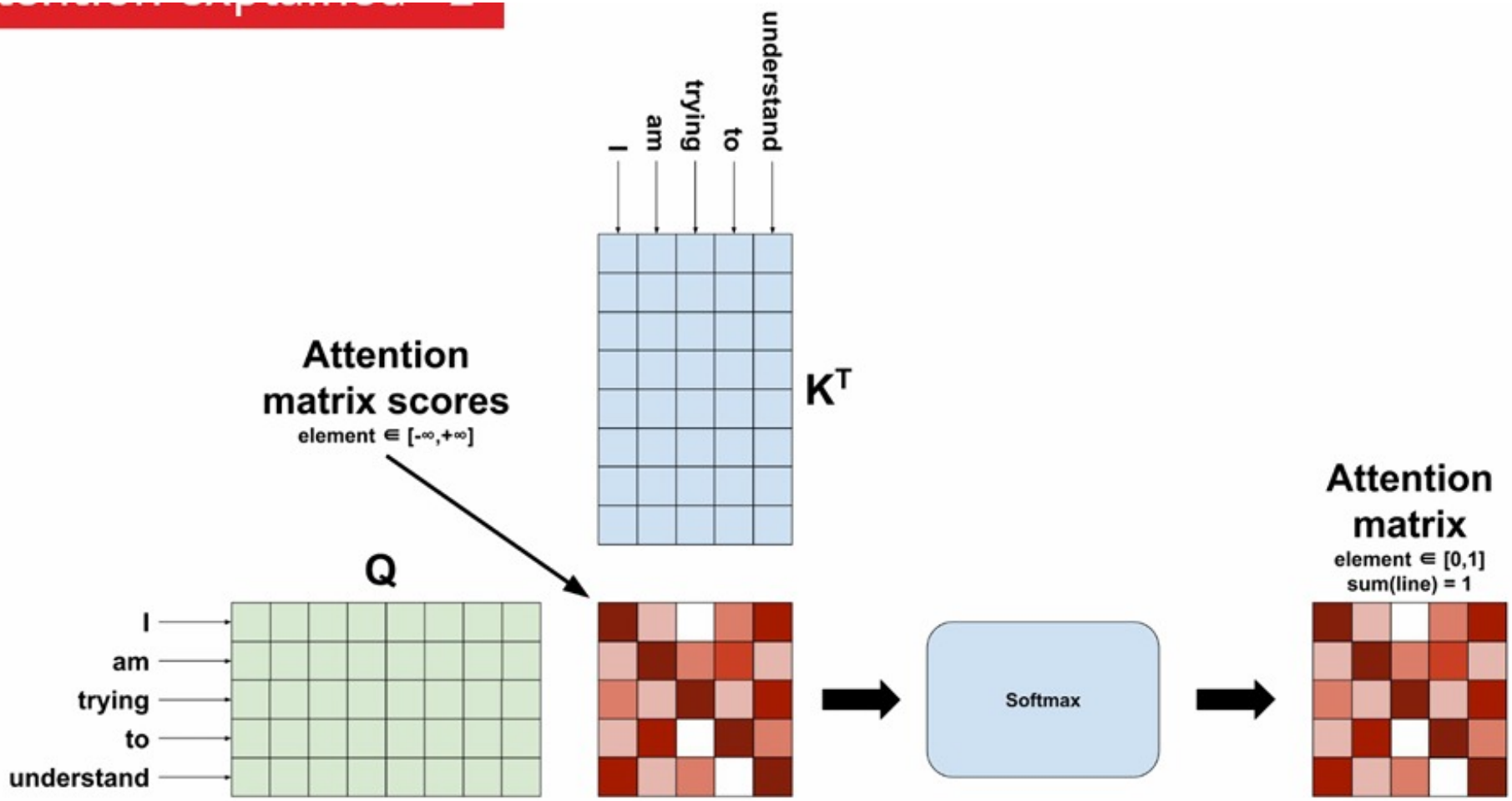


Figure: Explication visuelle de l'Attention

L'Encodeur : Multi-Head Attention

Principe

Lire la phrase sous différents "angles de vue" simultanément. Plutôt que d'une seule Attention, le modèle en utilise h (ex. $h = 8$) en parallèle.

- Chaque tête apprend à se concentrer sur des **types de relations différents** (syntaxe, coréférences, dépendances à longue distance).

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, ..., \text{head}_h)W^O$$

où $\text{head}_j = \text{Attention}(QW_j^Q, KW_j^K, VW_j^V)$.

Utilité : Enrichir la représentation contextuelle en capturant **plusieurs aspects du contexte** sous plusieurs angles.

L'Encodeur : Add & Norm

1. Add : La Connexion Résiduelle

$$\mathbf{z} = \mathbf{x} + \text{Sublayer}(\mathbf{x})$$

2. Norm : La Normalisation des Couches

$$\text{LayerNorm}(\mathbf{z}) = \gamma \odot \frac{\mathbf{z} - \mu}{\sigma} + \beta$$

Utilité Globale

- **Connexions Résiduelles (Add) :** Assurer un **chemin de gradient direct** ; Entraîner efficacement des **réseaux profonds**.
- **Normalisation (Norm) :** Maintenir les **activations stables** ; Accélérer la **convergence** de l'apprentissage.

L'Encodeur : Réseau Feed-Forward (FFN)

Objectif : Transformer et enrichir chaque token individuellement après l'étape d'Attention.

- Deux couches linéaires séparées par une activation **ReLU**.
- Les poids (W_1, W_2, b_1, b_2) sont les **mêmes** pour toutes les positions de la séquence.

Soit $\mathbf{x}$ le vecteur d'entrée à une position donnée :

$$\text{FFN}(\mathbf{x}) = \max(0, \mathbf{x}W_1 + b_1)W_2 + b_2$$

- **Non-Linéarité :** L'activation $\max(0, \cdot)$ (ReLU) permet au modèle de capturer des motifs complexes.
- **Réduction :** La seconde projection W_2 ramène la dimension à d_{model}.

Le Décodeur : Structure

Le Décodeur est une **pile de blocs identiques** (Layer), chacun contenant trois sous-couches :

1. **Self-Attention Masquée (Masked Multi-Head Attention)**
2. **Attention Encodeur–Décodeur (Cross-Attention)**
3. **Feed-Forward Network (FFN) + Add & Norm**

Processus : Le Décodeur fonctionne de manière **auto-régressive** (il boucle jusqu'à générer le jeton de fin de phrase).

Objectif : Générer séquentiellement la phrase cible, mot par mot, en utilisant la compréhension fournie par l'Encodeur.

Le Décodeur : Self-Attention Masquée

Formulation Mathématique

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T + M}{\sqrt{d_k}}\right) V$$

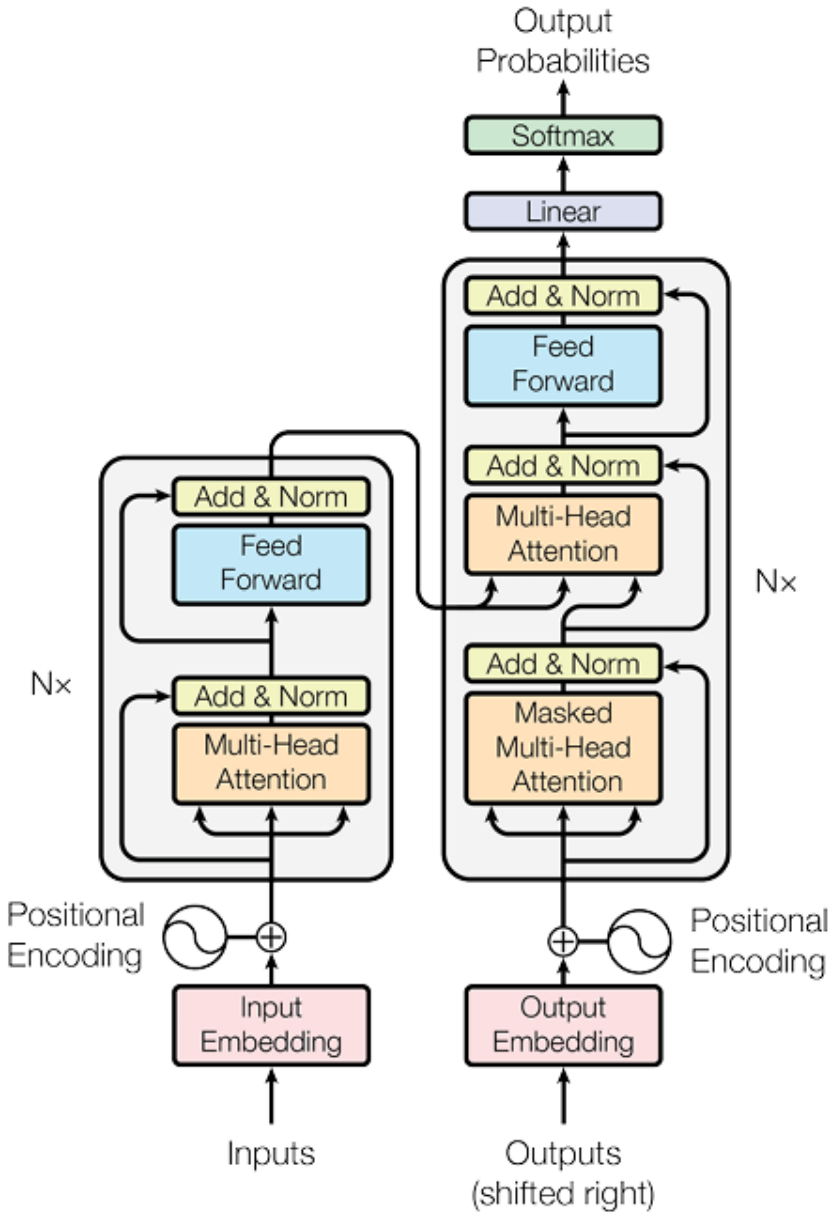
où M est la matrice de masquage :

$$M_{t,k} = \begin{cases} 0 & \text{si } k \leq t \\ -\infty & \text{si } k > t \end{cases}$$

Objectif

- Empêcher le modèle de "tricher" en regardant les mots futurs.
- Garantir que la prédiction à l'instant t dépend uniquement de la sortie $\mathbf{x}_1, \ldots, \mathbf{x}_t$.

Architecture du Transformer



Le Décodeur : Projection Finale et Génération

1. Projection Linéaire

- La sortie du Décodeur (vecteur d_{model}) est projetée sur la dimension du **Vocabulaire** (V_{size}).
- **Résultat :** Un vecteur de scores bruts (**logits**), un score pour chaque mot possible.

2. Softmax

- Appliqué aux logits pour convertir les scores en **probabilités**.

$$\text{Probabilité}(w_i) = \frac{e^{logit_i}}{\sum_{j=1}^{V_{\text{size}}} e^{logit_j}}$$

Processus Final

- Le token le plus probable est sélectionné pour la génération.
- Le cycle Décodeur est **répété** (auto-régressif) jusqu'à la fin de phrase ($\langle \text{EOS} \rangle$).

Conclusion : Impact des Transformers

Points Clés

- Les Transformers suppriment la récursivité des RNN/LSTM, utilisant l'attention pour capturer les dépendances à longue distance.
- L'architecture encodeur-décodeur et le mécanisme de Multi-Head Attention permettent un apprentissage parallèle et efficace.
- Positional Encoding assure la compréhension de l'ordre des mots dans les séquences.
- Résultats impressionnants en traduction automatique, NLP et reconnaissance vocale.

Ouverture

- Les Transformers sont à la base des modèles modernes comme BERT, GPT et autres LLM.
- Recherche continue : efficacité, réduction de taille, adaptation à différents types de données.